高级语言程序设计

第5章：多态、接口及内部类

计算机学院 陈建全
Email: cjq@nwpu.edu.cn

2024/2/8

- 5.1 多态
- 5.2 抽象类、接口
- *5.3 内部类

5.1 多态

- 本章重点讲述动态多态。
- 多态性是面向对象编程语言的一个重要特征，它将接口与实现分离的提高到另一个维度，将“是什么”和“如何做”分离。
- 多态性可以改进代码的组织，并提高了程序的可扩展性。
- 多态性处理的是类型的解耦。
- 多态也称为动态绑定、后期绑定或运行时绑定。

5.1.1 类型转换

- 对象可以表达为多个类型，即对象除了表达为自身的类型外，也可以表达为其基类型。
  - 比如战斗机也是飞机
  - 红苹果也是苹果
  - 比亚迪汽车也是汽车。
- 将对象的引用表达为自身的基类类型称为向上类型转换(Upcasting)。
- 有继承关系的引用变量之间的类型转换(type casting)规则
  - 向上转型是将子类的引用赋值给父类引用的过程，它是隐式的，无需显式转换 。向上转型是安全的，因为子类对象总是也是一个父类的实例。
  - 向上类型转换可能会“缩小”接口, 但是不能比父类的接口少。
  - 如果父类引用指向的是从子类向上转型来的对象，可以再次向下转型为子类类型。(Downcasting)。
  - 向下转型必须是显式的，并可能导致运行时错误（ClassCastException）。为了安全进行向下转型，可以先使用 instanceof 操作符检查类型
- 没有继承关系的引用变量之间进行塑性，会发生编译错误。

5.1.1 类型转换

◆看一个向上类型转换的例子（E0501Flight)

- ■枚举类型Attitude定义了三种飞行姿态。
- ■Plane类的方法flying表示飞行，它接收Attitude类型的参数。

```
//: polymorphism/flight/Attitude.java
package polymorphism.flight;
public enum Attitude {
    CRUISE, DIVE, LIFT; // 巡航、俯冲、起飞.
}
```

```
//飞机类: polymorphism/flight/Plane.java
package polymorphism.flight;
public class Plane {
    public void flying(Attitude n) {
        System.out.println("Plane.flying");
    }
}
```

5.1.1 类型转换

◆Fighter继承了Plane, 并且覆盖（Override)了flying方法。

◆Figher还追加了自己的方法fighting()。

```
//战斗机类: polymorphism/flight/Fighter.java
package polymorphism.flight;
//战斗机是飞机的一种，它们具有相同的接口:
public class Fighter extends Plane {
    // 重新定义接口方法:
    @Override
    public void flying(Attitude n) {
        System.out.println("Fighter.flying() " + n);
    }
    public void fighting() {
        System.out.println("Fighting...");
    }
}
```

5.1.1 类型转换

◆Flight.aviate()方法的形参 i 是类型为 Plane 的引用变量，但是它也接受 Plane 的派生类的对象。

◆调用aviate()时，实参类型为子类类型，无须显式转换。

```
//: polymorphism/flight/Flight.java
package polymorphism.flight;
public class Flight{
    public static void aviate(Plane i) {
        i.flying(Attitude.LIFT);
    }
    public static void main(String[] args) {
        Fighter fter = new Fighter();
        aviate(fter); //类型的向上转换
    }
}
```

```
//也可以这样:
Plane p = new Fighter();
aviate(p);
```

```
Plane p = new Fighter();//类型的向上转换
aviate(p);
```

◆main方法中也可以直接把战斗机类型赋值给类型变量p。

5.1.1 类型转换

◆向上类型转换可能会“缩小”接口, 但是不能比父类的接口少。

■如果Fighter类中把, 把flying的访问控制修改为proteced, 就会发生编译错误。

```
Cannot reduce the visibility of the inherited
method from PlaneJava(67109273)
```

■虽然对象本身包含有fighting()方法，但是通过Plane类型的引用变量，fighting()方法是无法访问的。

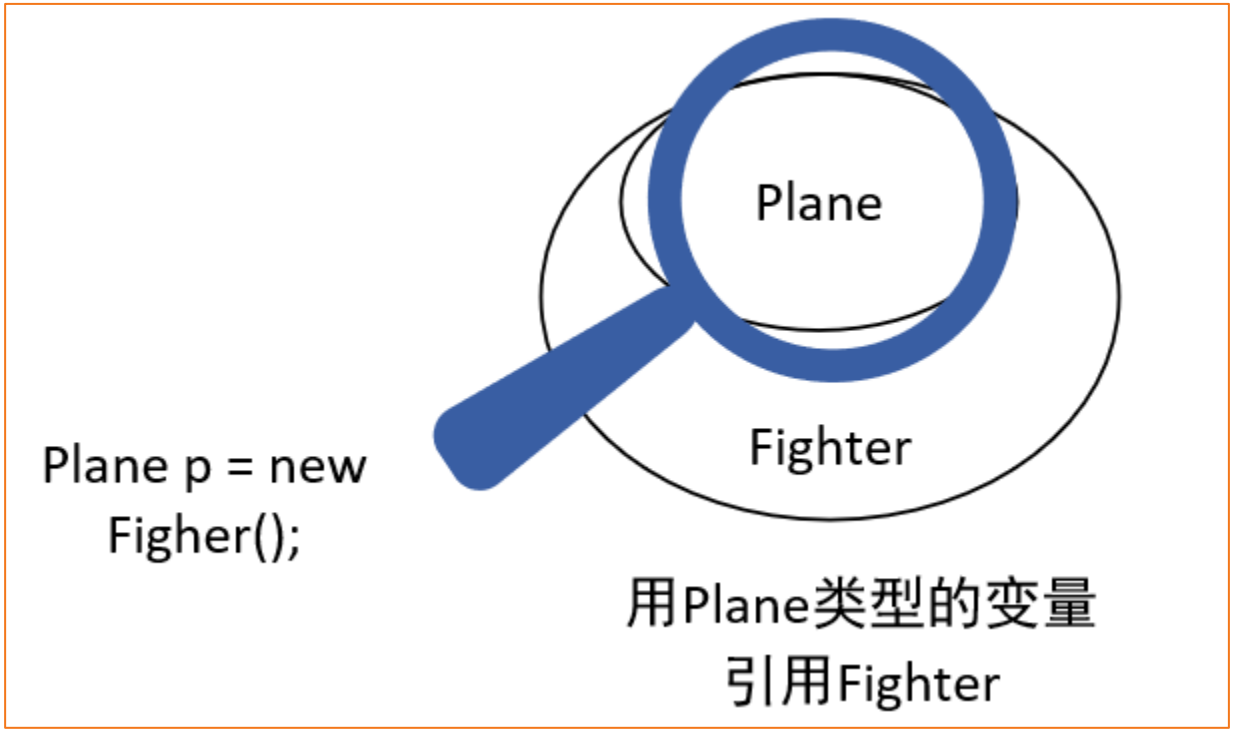
```
Plane p = new Fighter();

//编译错误: The method fighting() is undefined for the type
PlaneJava(67108964)
//p.fighting();
```

5.1.1 类型转换

◆向上类型转换可能会“缩小”接口, 但是不能比父类的接口少。

■编译器通过类Plane，无法识别fighting()方法。



5.1.1 类型转换

◆如果父类引用指向的是从子类向上转型来的对象，可以再次向下转型为子类类型。(Downcasting)。

- p指向Fighter,所以再次转型为Fighter没有问题。
- 转型后就可以调用Fighter新增的方法fighting()了。
- 在更复杂的场景下，为了安全进行向下转型，先使用 instanceof 操作符检查类型：

```
    public static void main(String[] args) {
        Plane p = new Fighter();
        //编译错误: The method fighting() is undefined for the
type PlaneJava(67108964)
        //p.fighting();
        if (p instanceof Fighter) {
            Fighter f =(Fighter)p;
            f.fighting();
        }
    }
```

5.1.1 类型转换

◆增加Bomber类，它是Plane的子类，Fighter的兄弟类。

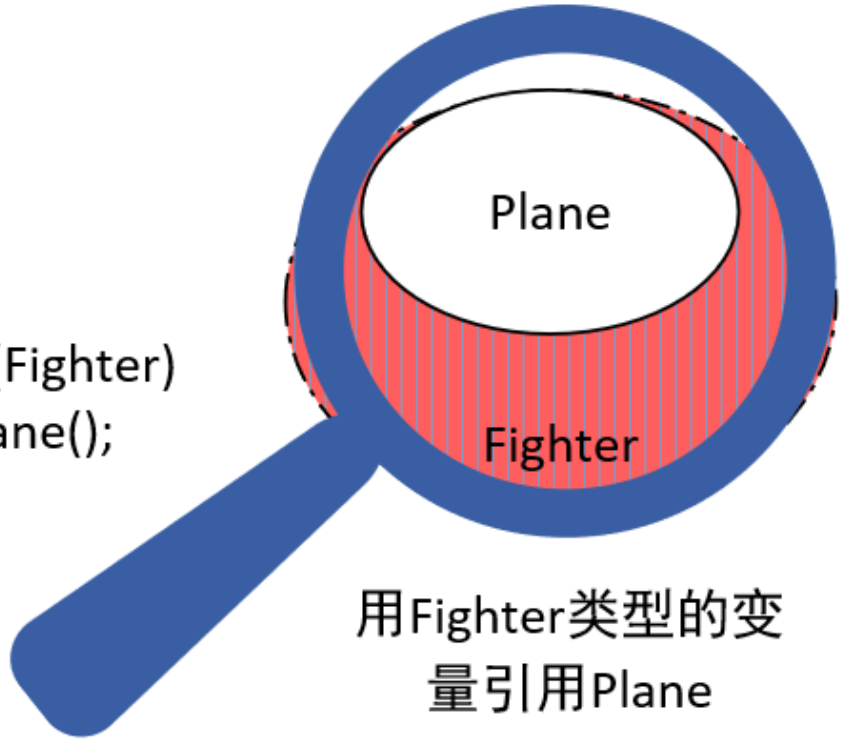
```
//轰炸机类: polymorphism/flight/Bomber.java
package polymorphism.flight;
public class Bomber extends Plane {
    // 重新定义接口方法:
    public void flying(Attitude n) {
        System.out.println("Bomber.flying() " + n);
    }
    public void bombing() {//轰炸机
        System.out.println("Bomber.bombing() ");
    }
}
```

5.1.1 类型转换

◆向下转型必须是显式的，并可能导致运行时错误（ClassCastException）。

- ■如下代码，因为Plane是Fighter的父类，所以编译没有问题。
- ■但是飞机无法转型为战斗机，运行时发生ClassCastException

```
Plane p = new Plane();
Fighter f =(Fighter)p;
```



5.1.1 类型转换

- 向下转型必须是显式的，并可能导致运行时错误（ClassCastException）。
  - 如下代码，因为Plane是Bomber的父类，所以编译没有问题。
  - 但是战斗机无法转型为轰炸机，运行时发生ClassCastException

```
Plane p = new Fighter();
Bomber b = (Bomber)p;
b.bombing();
```

```
Exception in thread "main" java.lang.ClassCastException: class
polymorphism.flight.Fighter cannot be cast to class
polymorphism.flight.Bomber (polymorphism.flight.Fighter and
polymorphism.flight.Bomber are in unnamed module of loader 'app')
        at polymorphism.flight.Flight3.main(Flight3.java:9)
```

5.1.1 类型转换

◆没有继承关系的引用变量之间进行塑性，会发生编译错误。

```
Bomber b = (Bomber)p;
Fighter f =(Fighter)b;
```

```
编译错误: Cannot cast from Bomber to FighterJava(16777372)
```

◆除了引用变量之间的类型转换，接口和实现了接口的类之间也可以进行类型转换。

5.1.2 形参与实参忽略对象类型

- 如果方法的形参是一个类的引用，那么实参可以是这个类的任何对象或者这个类的子类的对象。这是多态性的体现。
- 如果形参是一个接口类型，则任何实现了该接口的类的对象都可以作为实参。
- 很多情况下，方法内部可以不关心实参的真正类型，只要对形参类型编码即可。
  - 某些情况下，方法内部可能仍然需要知道实参的具体类型。在这些情况下，可以使用instanceof运算符来判断 实例的具体类型，以执行不同的操作。比如前面讲过的equals方法。
- 比如： aviate 的形参类型为Plane,调用时传入的实参是Fighter. 这就是类型向上转换时，形参与实参忽略对象类型。
- 这个特性使得代码简洁、利于扩展。

5.1.2 形参与实参忽略对象类型

◆反之，如果aviate的参数不是Plane类型，而是Fighter类型。

那么，对于轰炸机，则不得不重载一个新的方法，参数类型为Bomber。

◆而且重载的方法除了参数类型，方法体是一样的。

代码不简洁。

```
/* polymorphism/flight/flight5.java
* 覆盖方法来代替向上转换 */
package polymorphism.flight;
public class Flight5 {
    public static void aviate(Fighter i) {
        i.flying(Attitude.LIFT);
    }
    public static void aviate(Bomber i) {
        i.flying(Attitude.LIFT);
    }
    public static void main(String[] args) {
        Fighter figt = new Fighter();
        Bomber bob = new Bomber();
        aviate(figt); // 无需类型的向上转换
        aviate(bob);
    }
}
```

5.1.2 形参与实参忽略对象类型

- 每增加一个飞机类型，就要追加重载一个对应的aviate方法。而且除了方法参数，代码还是与已经存在的aviate方法重复的。
- 如果编写一个函数，它以一个基类为参数而不是任何特定派生类， 就可以解决因为追加子类带来的问题。
  - 利用Plane作为aviate的参数类型，就没有上述这些问题。

5.1.3 函数调用的绑定

◆动态绑定和静态绑定

- ■将函数调用连接到函数体的过程称为绑定。
- ■在程序运行之前进行绑定，它被称为早期绑定(early binding)：，即由编译器和链接器在编译和链接的时候绑定。又叫静态联编(Static Binding)。
  - □Java的private, static, final方法都是静态绑定。静态绑定效率较高。
  - □构造方法也是静态绑定。
  - □对于方法重载，首先是静态联编决定方法签名。
- ■后期绑定(late binding)，即程序在运行时，依据对象的类型来选择具体的函数体。后期绑定也称为动态绑定(dynamic binding)或运行时绑定(runtime binding)
  - □Java的方法覆盖就是动态绑定最好的例子。

5.1.3 函数调用的绑定

- 问题：在Plane作为aviate的形参，Fighter作为实参，运行结果什么样子的？从下面执行结果可以看到:
- 虽然aviate的参数类型是Plane, 但是由于实参是Fighter, i.flying()执行的是Fighter的对应方法，而不是Plane的对应方法。这是因为Java内部的"动态绑定"机制发挥了作用。
- 多态性允许我们根据父类编程，这样即使传入的是子类实例，程序也能正常运作。这同样遵循了面向对象编程中的一个核心原则——“面向接口编程，而不是面向实现编程”。因此，对于 aviate 方法，只需针对基类 Plane 编写正确的代码，无论是传入 Plane 的实例还是其任何子类的实例，都可以获得预期的行为。"

```
public static void aviate(Plane i) {
    i.flying(Attitude.LIFT);
}
public static void main(String[] args) {
    Fighter fter = new Fighter();
    aviate(fter); //类型的向上转换
```

输出结果:
Fighter.flying() LIFT

5.1.3 函数调用的绑定

- 运行时多态: 运行时调用的实例方法，由引用对象自身的类型决定，而不是引用变量的类型决定。aviate中，i的类型是Plane，但是运行的是Fighter类的flying()。
- 实例方法的覆盖：在aviate中，运行时没有调用Plane类的flying, Plane类的flying()被Fighter的flying()“覆盖”了。
- 后期绑定：实例方法的运行，不是编译时绑定的，而是运行时绑定的。

```
public static void aviate(Plane i) {
    i.flying(Attitude.LIFT);
}
public static void main(String[] args) {
    Fighter fter = new Fighter();
    aviate(fter); //类型的向上转换
```

输出结果:

```
Fighter.flying() LIFT
```

5.1.4 正确的函数调用

- 实例方法的调用是通过后期绑定以多态方式发生的。
- 在编写程序时，可以采用向基类引用发送消息的方式，让所有派生类对象都使用相同的代码工作。
- 运行时会根据实参的类型，后期绑定找到正确的子类的方法体。
- 这样，基类定义的方法只需要撰写一次，而由各个子类提供具体的实现细节，从而增加了代码的可重用性和扩展性。
- 用一下的Shape类作为基类，演示动态绑定：

```
//: polymorphism/shape/Shape.java
package polymorphism.shape;
public class Shape{
    public void draw(){
     }
    public void earse(){
    }
}
```

5.1.4 正确的函数调用

◆Shape有子类Circle, Square, Triangle。

```
//: polymorphism/shape/Circle.java
package polymorphism.shape;
public class Circle extends Shape{
    public void draw(){
        System.out.println("draw Circle");
    }
    public void earse(){
        System.out.println("earse Circle");
    }
}
```

```
//: polymorphism/shape/Square.java
package polymorphism.shape;
public class Square extends Shape{
    public void draw(){
        System.out.println("draw Square");
    }
    public void earse(){
        System.out.println("earse Square");
    }
}
```

5.1.4 正确的函数调用

◆Shape有子类Circle, Square, Triangle。

```
//: polymorphism/shape/Triangle.java
package polymorphism.shape;
public class Triangle extends Shape{
    public void draw(){
        System.out.println("draw Triangle");
    }
    public void earse(){
        System.out.println("earse Triangle");
    }
}
```

5.1.4 正确的函数调用

◆RandomShapeGenerator 是一种“工厂”，每次调用其next()方法时都返回一个Shape 对象的引用。但是，在 next 函数体中，会随机产生 Circle、Square 或Triangle。

```
//: polymorphism/shape/RandomShapeGenerator.java
package polymorphism.shape;
import java.util.*;
public class RandomShapeGenerator {
    private Random rand = new Random();
    public Shape next() {
        switch(rand.nextInt(3)) {//依据随机值决定创建哪个子类对象
            default:
            case 0: return new Circle();
            case 1: return new Square();
            case 2: return new Triangle();
        }
    }
}
```

5.1.4 正确的函数调用

◆main()中首先用随机生成的对象填充数组，然后分别调用每个对象的draw()来显示。

```
//: polymorphism/Shapes.java
import polymorphism.shape.*;
public class Shapes {
    private static RandomShapeGenerator gen =new
RandomShapeGenerator();
    public static void main(String[] args) {
        Shape[] s = new Shape[9];
        // 使用 shapes 类型填充数组:
        for(int i = 0; i < s.length; i++)
            s[i] = gen.next();
        // 实现多态方法调用:
        for(Shape shp : s)
            shp.draw();
    }
}
```

5.1.4 正确的函数调用

- main中不必关心具体的对象是哪个子类，只需要针对每个数组元素（类型为Shape)发送draw()消息。
- 数组中每个对象收到draw消息之后，根据自己的类型，产生不同的响应，也就是输出的结果是根据对象类型变化的。（多态）
- 将来即使追加新的类型，比如Rectangle, main方法中的代码不必修改，仍然能够正确执行。
- 为了调用RandomShapeGenerator的实例方法next，在Shapes类中定义并new了一个类变量gen.

思考： RandomShapeGenerator的next能否修饰为static?

```
draw Square
draw Square
draw Circle
draw Triangle
draw Circle
draw Triangle
draw Circle
draw Square
draw Circle
```

5.1.5 可扩展性

- 在飞机 Plane 例子中，可以不更改 aviate()方法向程序中添加任意数量的新的 Plane 类型。
- 在一个设计良好的面向对象程序中，基本上所有方法都遵循类似 aviate()函数的方式，并且只与基类接口通信。
  - 这样的程序是可扩展的，因为可以通过从公共基类继承新数据类型来添加新功能。操作基类接口的方法根本不需要更改就可以适应新类。
- 如果以 Plane 为基类，添加更多的新类，所有这些新类都可以与旧的、未更改的 aviate()方法一起正常工作。
- 由于 aviate()位于单独的文件中，即使在 Plane 的接口中添加了新方法，那么，aviate的编译单元不重新编译，aviate()仍然可以正常工作。
- (E0503Flight.java)

5.1.5 可扩展性

- 在基类Plane中追加了adjust和what方法。Fighter, Scout和Bomber都覆盖了adjust和what方法。
- 新增Fighter子类J17和J15.
- J17覆盖了adjust但是没有覆盖what
- J15覆盖了what但是没有覆盖adjust
- 未覆盖的方法会执行父类的。
- avaite没有做修改，结果仍然正确。

```
aviateAll(airplane );
for (Plane p : airplane) p.adjust();
for (Plane p : airplane)
System.out.println(p.what());
```

```
Fighter.flying() DIVE
scout.flying() DIVE
Bomber.flying() DIVE
J7.flying() DIVE
J15.flying() DIVE
Adjusting Fighter
Adjusting scout
Adjusting Bomber
Adjusting J7
Adjusting Fighter
Fighter
scout
Bomber
Fighter
J15
```

5.1.6 多态使用中的知识点

- （1）“覆盖”私有方法
- private和final的实例方法都是静态绑定，不能被覆盖。
- private同时也是对子类隐藏的。不能被子类中的方法覆盖。
- 试图覆盖私有函数时，编译器不会提示警告。在编写派生类的函数时，函数名尽量不与私有基类的方法名同名。

5.1.6 多态使用中的知识点

- ◆比如如下代码，运行结果表明没父类f()方法没有被覆盖。
- ◆除非使用了@Override注解，否则没有编译错误。

```
public class E0504PrivateOverride {
    private void f() { System.out.print("private f()"); }
    public static void main(String[] args) {
        E0504PrivateOverride po = new Derived();
        po.f();
    }
}
class Derived extends E0504PrivateOverride {
    //@Override
    public void f() { System.out.print("public f()"); }
}
```

输出结果:

```
private f()
```

5.1.6 多态使用中的知识点

- （2）字段的访问是编译时解析的，因此不是多态的。
- 如果有字段隐藏，访问的被字段由引用变量的类型决定。
- 通常会将所有字段设为私有，不去直接访问它们。另外，为基类成员变量和派生类成员变量给出不同的标识符，避免造成混淆。

5.1.6 多态使用中的知识点

◆对于同一个Child对象，输出的字段由引用变量决定：

■用Parent类型obj输出的是“Parent", Child类型c输出的是"Child"

```
Parent
Child
```

```
class Parent {
    String name = "Parent";
}
class Child extends Parent {
    String name = "Child";
}
public class E0505FieldHidden {
    public static void main(String[] args) {
        Parent obj = new Child();
        System.out.println(obj.name);  // 输出 "Parent"
        Child c = (Child)obj;
        System.out.println(c.name); //输出Child;
    }
}
```

5.1.6 多态使用中的知识点

- (3) 静态方法不是多态的。
- static方法属于类本身，而不属于类的实例。
- Static方法也是静态绑定的。当调用一个静态方法时，这个调用是根据声明的引用类型来解析的，而不是实际对象的类型。

5.1.6 多态使用中的知识点

◆obj.display() 它会调用 Base.display()，而不是 Derived.display()，这是因为静态方法不是多态的，这里的调用根据引用类型Base来解析

Display in Base

```
class Base {
    static void display() {
        System.out.println("Display in Base");
    }}
class Derived extends Base {
    static void display() {
        System.out.println("Display in Derived");
    }}
public class E0506StaticPolymorphism {
    public static void main(String[] args) {
        Base obj = new Derived();
        obj.display();
    }}
```

5.1.7 构造函数和多态性

- 构造函数实际上是静态方法，其静态声明是隐式的。
- 构造函数的主要职责是执行对象的初始化操作，这包括为实例变量分配初始值以及执行必要的设置或创建工作，以确保对象在使用前处于一个合理的状态
- 在派生类的构造过程中，通过继承关系将子类、父类甚至祖先类链接起来，调用子类构造函数时，亦调用父类的构造函数。构造函数有一项特殊的功能是查看对象是否正确构建。
- 派生类只能初始化自己的成员，不能初始化基类的成员。因此，必须调用所有构造函数，否则不会构造整个对象。这就是编译器对派生类的每个部分都强制执行构造函数调用的原因。
- 如果没有在派生类构造函数体中显式调用基类构造函数，它将调用缺省的构造函数。如果没有缺省构造函数，编译器会报错。
- 另外，在类没有定义构造函数的情况下，编译器会自动合成一个默认构造函数。

5.1.7 构造函数和多态性

- （1）构造函数调用顺序
- 构造函数调用的顺序很重要。
- 继承时，了解基类的所有信息，并且可以访问基类的任何公共和受保护成员。
- 因此，派生类构造函数主体执行前，需要确保基类的所有成员都已初始化。保证这一点的唯一方法是首先调用基类构造函数。
- 每一层构造函数的执行顺序是：
  - 成员变量和实例初始化块：在调用任何构造方法的代码之前，该类的成员变量初始化和实例初始化块（如果有的话）将按照它们在代码中声明的顺序执行。
  - 构造方法主体：类的构造方法的主体代码最后执行。
- 总之，层层负责，确保所有实例变量都设置了恰当的初值。而且下层初始化代码执行之前，确保上层初始化完成。因为下层初始化可能需要引用上层的变量。

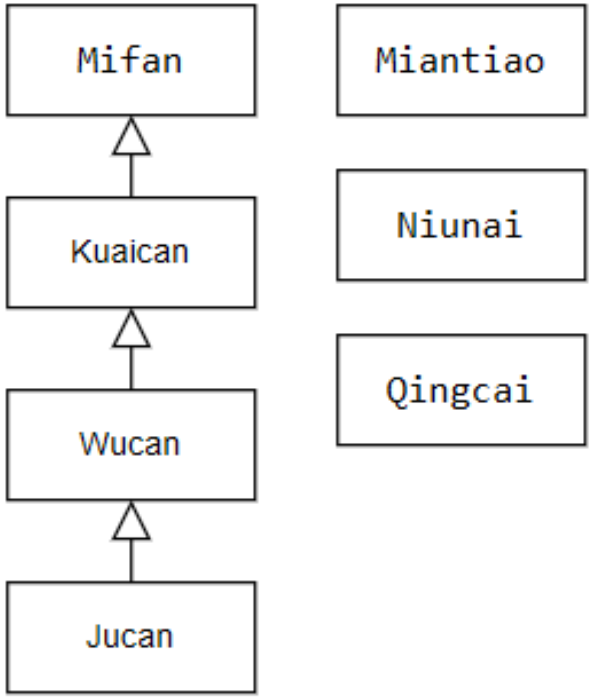
5.1.7 构造函数和多态性


```
class Mifan {
    Mifan() { System.out.println("Mifan()"); }
}
class Miantiao {
    Miantiao() { System.out.println("Miantiao()"); }
}
class Niunai {
    Niunai() { System.out.println("Niunai()"); }
}
class Qingcai {
    Qingcai() { System.out.println("Qingcai()"); }
}
class Kuaican extends Mifan {
    Kuaican() { System.out.println("Kuaican()"); }
}
class Wucan extends Kuaican {
    Wucan() { System.out.println("Wucan()");}
}
```

◆执行顺序: 父类构造函数->本类实例初始化->本类构造函数

```
public class E0507Jucan extends Wucan {
    private Miantiao b = new Miantiao();
    private Niunai c = new Niunai();
    private Qingcai l = new Qingcai();
    public E0507Jucan() { System.out.println("Jucan()"); }
    public static void main(String[] args) {
        new E0507Jucan();
    }
}
```

```
Mifan()
Kuaican()
Wucan()
Miantiao()
Niulai()
Qingcai()
Jucan()
```



- （2）继承和对象删除
- 当创建的对象不需要被程序其余部分使用时，即一个对象的引用不存在时，大多数时候不需考虑对象的删除；Java 的垃圾收集器会在一定的条件满足时删除无用的对象。
- 当然，如果用户需要自己完成一些清理工作，就需要为新类创建一个 dispose()函数（函数名称用户自己指定）。在需要清理时调用dispose()。
- 对于继承，如果有任何进行的特殊清理，则必须覆盖派生类中的 dispose()。当在继承类中重写 dispose()时，记住调用 dispose()的基类版本是很重要的，否则不会发生基类的清理工作。

◆（2）继承和对象删除

◆dispose方法的架构如下：

```
protected void dispose() {
    //进行本类的清理工作，必要时调用成员变量的dispose();
    //调用父类的dispose()
    super.dispose();
}
```

◆确保清理顺序和构造顺序相反，以避免清理中使用到已经被清理的资源。

5.1.8 构造函数中的多态方法*

- 在构造方法内调用正在构造的那个对象的动态绑定方法
  - 会调用位于派生类里的一个方法
  - 被调用方法要操纵的成员可能尚未得到正确的初始化
  - 可能造成一些难于发现的程序错误
- 避免在构造函数中调用可以被子类覆写（Override）的方法。
- 在构造函数中调用的唯一安全方法是本类或基类中的 final 函数或者 static 修饰的函数。

5.1.8 构造函数中的多态方法*


```
class Painting {
    void draw() { System.out.print("Painting.draw()"); }
    Painting() {
        System.out.println("Painting() before draw()");
        draw();
        System.out.println("Painting() after draw()");
    }
}
class WaterPainting extends Painting {
    private int sizes = 1;
    WaterPainting(int r) {
        sizes = r;
        System.out.println("WaterPainting.WaterPainting(),
sizes = " + sizes);
    }
    void draw() {
        System.out.println("WaterPainting.draw(), sizes = " +
sizes);
    }
}
```

5.1.8 构造函数中的多态方法*

- 在 WaterPainting 类中， Painting.draw()被覆盖。
- 但是 Painting 构造函数调用了这个方法，调用时使用了多态 WaterPainting.draw( )。
- 但是，当 Painting 的构造函数调用draw()时，sizes 没有使用其默认的初始值 1，而是变量初始化值 0。这说明子类对象还没有完全产生。

```
public class E0508PolyConstructor {
    public static void main(String[] args) {
        new WaterPainting(5);
    }
}
```

```
Painting() before draw()
WaterPainting.draw(), sizes = 0
Painting() after draw()
```

5.1.9 协变返回类型*

- 在子类覆盖父类方法时，子类方法的返回类型可以是父类方法返回类型的子类型。
- 协变返回类型的支持是从Java 5开始的。允许方法覆盖不仅仅可以在行为上提供更具体的实现，还能在返回类型上提供更具体的类型。这样可以提高API的可用性和类型安全性。

5.1.9 协变返回类型*


```
class Automobile {
    public String toString() { return "Automobile"; }
}
class BYD extends Automobile {
    public String toString() { return "BYD"; }
}
class Factory {
        Automobile process() { return new Automobile(); }
}
class BYDFactory extends Factory {
    BYD process() { return new BYD(); }
}
public class E0508CovariantReturn {
    public static void main(String[] args) {
        Factory m = new Factory();
        Automobile g = m.process();
        System.out.println(g);
        m = new BYDFactory();
        g = m.process();
        System.out.println(g);
    }
}
```

```
Automobile
BYD
```

5.1.10 继承设计

- 过度使用继承，会造成程序的复杂性。
- **组合/聚合复用原则（Composite/Aggregate Reuse Principle CARP）**

  尽量使用合成/聚合达到复用，尽量少用继承。

- 一般设计准则：使用继承来表达行为的差异，并使用字段来表达状态的变化

```
class Actor {
    public void act() {}
}
class HappyActor extends Actor {
    public void act() { System.out.println("HappyActor"); }
}
class SadActor extends Actor {
    public void act() { System.out.println("SadActor"); }
}
```

5.1.10 继承设计

- Stage 对象包含对 Actor 的引用。Actor 代表的是状态。
- SadActor和HappyActor通过继承Actor,代表不同的行为。
- change()通过改变Stage状态，同时也改变了Stage的行为。

```
class Stage {
    private Actor actor = new HappyActor();
    public void change() { actor = new SadActor(); }
    public void performPlay() { actor.act(); }
}
public class E0509PolyTest  {
    public static void main(String[] args) {
        Stage stage = new Stage();
        stage.performPlay();
        stage.change();
        stage.performPlay();
    }
}
```

5.1.10 继承设计

- Stage 对象包含对 Actor 的引用。Actor 代表的是状态。
- SadActor和HappyActor通过继承Actor,代表不同的行为。
- change()通过改变Stage状态，同时也改变了Stage的行为。

```
class Stage {
    private Actor actor = new HappyActor();
    public void change() { actor = new SadActor(); }
    public void performPlay() { actor.act(); }
}
public class E0509PolyTest  {
    public static void main(String[] args) {
        Stage stage = new Stage();
        stage.performPlay();
        stage.change();
        stage.performPlay();
    }
}
```

5.1.11 向下转换和运行时类型信息*

- 对象类型的向上转换丢失了特定类型信息，因此为了获得特定类型信息，就需要恢复原来的类型。
- 因为基类不能比派生类有更多的接口，所以向上转换总是安全的，通过基类接口发送的每条消息都能保证被接受。反之，如果并不真的知道一个对象（例如：形状）它是一个圆、三角形或正方形或其他类型，那么向下转换就可能出现无法接收的消息。
- 为了解决这个问题，必须有某种方法来保证向下转换是正确的，这样你就不会不小心转换到错误的类型，然后发送一个对象不能接受的消息。
- 在 Java 中，为了保证向下转换的正确性，每个转换都会被检查。即，在运行时会检查这个转换，如果类型不正确，抛出一个 ClassCastException。
- 这种在运行时检查类型的行为称为运行时类型识别（简称 RTTI）

5.1.11 向下转换和运行时类型信息*

◆子类MoreUseful追加了自己的方法：u(),v(),w()

```
//: polymorphism/RTTI.java
class Useful {
    public void f() {}
    public void g() {}
}
 class MoreUseful extends Useful {
    public void f() {}
    public void g() {}
    public void u() {}
    public void v() {}
    public void w() {}
}
```

5.1.11 向下转换和运行时类型信息*

- x[1].u()发生编译错误，因为数组元素类型是Useful.
- ((MoreUseful)x[0]).u()发生运行时错误，因为这个元素是父类类型，无法转换为子类类型（ClassCastException ）。

```
public class E0510RTTI {
    public static void main(String[] args) {
        Useful[] x = {
            new Useful(),
            new MoreUseful()
        };
        x[0].f();
        x[1].g();
        // Compile time: method not found in Useful:
        x[1].u(); //编译时该语句错误
        ((MoreUseful)x[1]).u(); // 合法向下转换
        ((MoreUseful)x[0]).u(); // 向下转换失败，抛出异常
    }
}
```

- 接口和抽象类是一种更高层次的将描述与实现分开的方法。
- 从描述与实现分开的角度看，普通类、抽象类、接口是逐步加强的。
- 抽象类是介于普通类和接口之间的一种中间步骤。
- 抽象类声明的语法形式为（其中Number是类名）。

```
abstract class Number {
    ...
}
```

- 抽象类不能实例化。
  - 如果写:
  - new Number();
  - 会发生编译错误。

◆抽象方法

- 声明的语法形式为

```
public abstract <returnType> <methodName>(...);
```

- 仅有方法头，而没有方法体和操作实现
- 抽象方法时不能使用 `private` 修饰，因为抽象方法必须被子类重写，而如果使用了 private 声明，则子类是无法重写的。
- 抽象类可以包含抽象方法
- 抽象方法必须存在于抽象类中；
- 具体实现由当前类的不同子类在它们各自的类声明中完成

5.2.1 抽象类

◆需注意的问题

- ■一个抽象类的子类如果不是抽象类，则它必须为父类中的所有抽象方法书写方法体，即重写父类中的所有抽象方法
- ■只有抽象类才能具有抽象方法，即如果一个类中含有抽象方法，则必须将这个类声明为抽象类
- ■除了抽象方法，抽象类中还可以包括非抽象方法

◆比如前面的例子，Plane类就可以定义为抽象类。

```
abstract class Plane {
    int i; //可以定义实例变量
    abstract void flying(Attitude n);
    String what() { return "Plane"; }
    abstract void adjust();
}
```

5.2.2 接口

◆接口

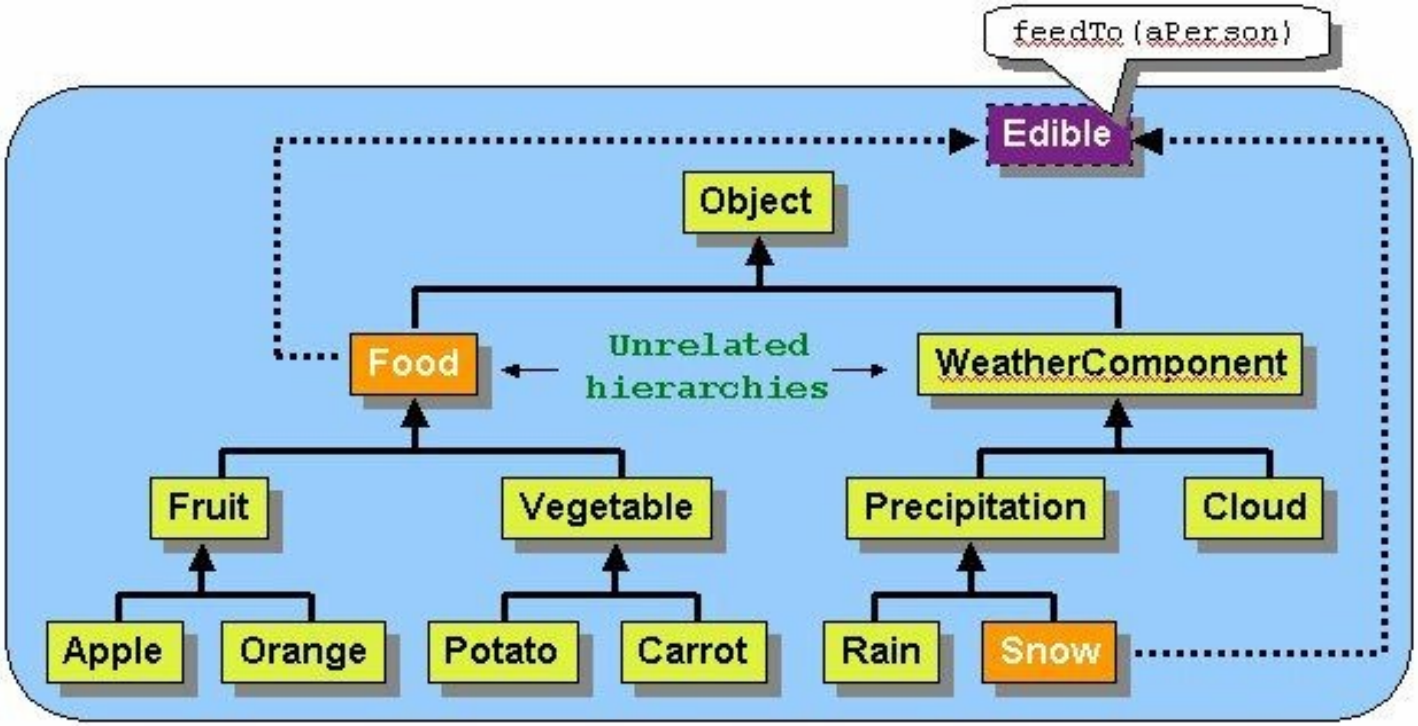
- 与抽象类一样都是定义多个类的共同属性
- 使抽象的概念更深入了一层，是一个“纯”抽象类，它只提供一种形式，并不提供实现
- 允许创建者规定方法的基本形式：方法名、参数列表以及返回类型，但不规定方法主体
- 也可以包含基本数据类型的数据成员，但它们都默认为static和final

◆接口的作用

- 是面向对象的一个重要机制
- 实现多继承，同时免除C++中的多继承那样的复杂性
- 建立类和类之间的“协议”
  - 把类根据其实现的功能来分别代表，而不必顾虑它所在的类继承层次；这样可以最大限度地利用动态绑定，隐藏实现细节
  - 实现不同类之间的常量共享

5.2.2 接口

◆接口允许我们在看起来不相干的对象之间定义共同行为



5.2.2 接口

◆接口的语法：

```
[public] interface <interface_name> [extends parent
interface_name] {
    [public] [static] [final] <type> var_name;
    [public] [abstract]<type> <method_name>(<parameter-
iist>);
}
```

◆interface 表示该定义为接口

◆<interface_name>表示接口的名称

◆var_name 代表一个静态成员变量名，<type>表示变量的类型。

◆<method_name>表示抽象方法名称，<parameter-list>表示方法参数列表，<type>表示抽象方法的返回值。

5.2.2 接口

- 接口的语法：
- 接口的访问控制符要么是public，要么没有。如果不使用public 关键字，接口具有包访问权限
- 变量的访问控制和修饰可以省略，所有变量都自动被视为public static final，即它们是公共的、静态的且不可变的。
- 方法的访问控制和修饰可以省略，这时方法就被视为public abstract的。

5.2.2 接口（Interface)*

- Java 8在接口中引入了两个重要的新概念：默认方法（Default Methods）和静态方法（Static Methods）。这些扩展赋予了接口更多的功能性，并允许接口拥有某些实现细节，而在此之前，接口只能包含抽象方法。
  - 默认方法 (Default Methods)
  - 默认方法允许在接口中包含具有实现体的方法。这意味着当一个类实现了这样的接口时，它可以继承默认方法的实现，也可以选择覆盖这个方法。默认方法使用default关键字标记。

```
public interface MyInterface {
    void abstractMethod(); // 传统的抽象方法，仍然被允许
    default void defaultMethod() {
        System.out.println("This is a default method");
    }
}
```

  - 默认方法的引入主要是为了使得接口能够向后兼容。当你需要在现有的接口中添加新的方法时，你可能不想强迫所有实现了该接口的类都去实现这个新方法。默认方法使得开发者可以提供方法的默认实现，而所有现有的实现类都会从接口中继承这个默认实现，不会导致现有代码的破坏。

5.2.2 接口（Interface)*

- 静态方法 (Static Methods)
- Java 8之后，可以在接口内部定义静态方法。静态方法必须提供一个实现体，并且它们不会被接口的实现类继承。这些方法可以通过接口名直接调用。

```
public interface MyInterface {
    static void staticMethod() {
        System.out.println("This is a static method");
    }
}
```

- 静态方法帮助开发者在接口中创建辅助方法，而不需要使用抽象类。

5.2.2 接口（Interface)*

- Java 9进一步扩展了接口的能力，允许在接口中使用私有方法和私有静态方法。这些方法不像默认方法和静态方法那样对继承它们的类或外部世界可见。
- 私有方法主要用于避免在接口中的多个默认方法之间出现代码重复，因为它们可以共享接口内部的公共代码。

```
public interface MyInterface {
    private void privateMethod() {
        System.out.println("This is a private method");
    }

    private static void privateStaticMethod() {
        System.out.println("This is a private static method");
    }
}
```

5.2.2 接口（Interface）

◆接口的实现

- 接口不能用new运算符直接产生对象，必须利用其特性设计新的类，再用新类来创建对象
- 利用接口设计类的过程，称为接口的实现，使用implements关键字
- 语法如下

```
public class 类名称 implements 接口名称 {
        /* Bodies for the interface methods */
        /* Own data and methods. */
}
```

- 除非抽象类，否则必须实现接口中的所有方法
- 被实现的接口的方法必须声明成public

5.2.2 接口（Interface）

◆前面例子中抽象类Plane修改为接口：
(E0512FlightInterface.java)

```
interface Plane {
    // 编译时常量定义:
    int VALUE = 5; // static & final
    // 不能包含方法体的定义:
    void flying(Attitude n); //隐含的访问特性是 public
    void adjust();
}
class Fighter implements Plane {
    public void flying(Attitude n) {
        System.out.println(this + ".flying() " + n);
    }
    public String toString() { return "Fighter"; }
    public void adjust() { System.out.println(this +
".adjust()"); }
}。。。。。。
```

5.2.3 Java 中的多重继承

◆多重继承

■Java的设计以简单实用为导向，不允许一个类有多个父类

■但允许一个类可以实现多个接口，通过这种机制可实现多重继承

■一个类实现多个接口的语法如下

```
[类修饰符] class 类名称 implements 接口1,接口2，...
{
... ...
}
```

◆下面看一个多重继承的例子。

5.2.3 Java 中的多重继承

◆Hero继承自ActionCharacter,并实现多个接口。

◆ActionCharacter 已经定义fight(), 并且与接口中的方法签名一致，因此Hero中可以不再定义fight()

```
interface CanFight {
    void fight();}
interface CanSwim {
    void swim(); }
interface CanFly {
    void fly();}
class ActionCharacter {
    public void fight() {}   }
class Hero extends ActionCharacter implements CanFight,
CanSwim, CanFly {
    public void swim() {}
    public void fly() {}
    public void act() {}
}
```

5.2.3 Java 中的多重继承

◆Hero的类型可以隐式向上转型为: ActionCharacter, CanFight, CanSwim, CanFly

```
public class E0513Adventure {
    public static void t(CanFight x) { x.fight(); }
    public static void u(CanSwim x) { x.swim(); }
    public static void v(CanFly x) { x.fly(); }
    public static void w(ActionCharacter x) { x.fight(); }
    public static void main(String[] args) {
        Hero h = new Hero();
        t(h); // 当作 CanFight
        u(h); // 当作 CanSwim
        v(h); // 当作 CanFly
        w(h); // 当作 ActionCharacter
    }
}
```

5.2.3 Java 中的多重继承

◆向上转型后，还可以再次向下转型为Hero。

◆转型后可以访问的成员，由引用变量类型决定。

```
Hero h = new Hero();
CanFight f = (CanFight)h;//对象转型为接口类型
f.fight(); //调用接口中方法
f.act(); //接口中没有声明此方法，不可以
h.act(); //类中有此方法，可以
((Hero)f).act(); //向下转型回hero,可以
```

◆为了避免转型失败，可以使用instanceof进行类型检查。

5.2.3 Java 中的多重继承

- 接口和抽象类的选择
- 使用接口有两个原因，一个是程序的灵活性，另外一个是防止客户端程序员创建此类的对象。
- 防止客户端程序员创建某个类时，可以使用接口，也可以使用抽象类。
- 尽可能地抽象。因此，更倾向使用接口而不是抽象类。只有当必要时才使用抽象类。比如需要提供一些实现或状态（即成员变量）,就使用抽象类。
- 但是接口和抽象类也不要过度使用，如果普通类能够很好地满足需要，就没有必要使用接口和抽象类。

5.2.3 Java 中的多重继承

- instancof:
- 在Java中，instanceof 运算符用于检查一个对象是否是特定类或接口的实例。它的一般用法是检查对象在转型之前是否属于目标类型，来避免在转型时发生 ClassCastException。
- 如果左侧的对象是 null，instanceof 结果总是 false，因为 null 不是任何类型的实例。
  - null instanceof Object总是false
- instanceof 不仅可以用于类，也可以用于接口。如果对象的类实现了指定的接口，那么 instanceof 检查也会返回 true。
- instanceof 运算符考虑到了类的继承层次结构。如果实例属于被检查类型的子类，那么 instanceof 检查也将返回 true。
- instanceof 也是运行时检查的，意味着它不是在编译时确定 结果，而是在程序运行时根据对象的实际类型来动态判断的。

5.2.3 Java 中的多重继承*

- *参考资料：*
- *RelationalExpression* instanceof *ReferenceType*

- At run time, the result of the instanceof operator is true if the value of the RelationalExpression is not null and the reference could be cast to the ReferenceType without raising a ClassCastException. Otherwise the result is false.
  - From 《The Java® Language Specification Java SE 10 Edition》

◆下面是例子instanceof结果全部是true:

```
interface t{
    void fly();
}
。。。。。。
    public static void main(String[] args) {
        Hero h = new Hero();
        CanFight f = (CanFight)h;//对象转型为接口类型
        f.fight(); //调用接口中方法
        ((CanFly)f).fly(); //从CanFight转型为CanFly可以
        ((t)f).fly(); //ClassCastException
        System.out.println(f instanceof Hero);//true
        System.out.println(f instanceof
ActionCharacter);//true
        System.out.println(f instanceof CanFight);//true
        System.out.println(f instanceof CanSwim);//true
        System.out.println(f instanceof CanFly);//true
    }  }
```

5.2.4 接口的继承

◆Java 接口的继承是面向对象编程中的一项特性，它允许一个接口继承另一个接口，从而使得子接口具备父接口的属性。在 Java 中，接口的继承使用 extends 关键字。

■继承基础：和类的继承类似，当一个接口继承另一个接口时，子接口会继承父接口中的所有抽象方法。

■多重继承：与类不同，Java 接口支持多重继承，即一个接口可以继承多个接口。这是因为接口仅仅定义了方法签名，没有具体的实现，所以不存在多重继承带来的歧义问题。

```
public interface InterfaceName extends Interface1,
Interface2, Interface3 {
    // 抽象方法
    // 默认方法
    // 静态方法
    // 常量定义
}
```

5.2.4 接口的继承

- 具体实现：继承接口的类或者继承自另一个接口的接口都必须提供所有接口方法的具体实现，除非该类是抽象的。
- 方法冲突：*
  - 在多继承时，如果一个接口从多个父接口继承同一个方法（即方法签名相同），并且父接口对这个方法的声明都是抽象的，那么子接口可以不用理会这个冲突，子接口中只会有一个这样的方法。
  - 如果多个继承的接口中有相同的默认方法，那么子接口需要覆盖这个默认方法，以解决冲突
- 默认方法*：从 Java 8 开始，接口可以有默认的方法实现（使用 default 关键字）。如果一个接口继承了另一个接口，并且两个接口中包含相同签名的默认方法，子接口需要提供这个方法的实现，或者明确指定使用哪一个父接口的默认实现

5.2.4 接口的继承

◆MountainTorrents继承了两个接口RainStorm和 Danger ，实现MountainTorrents需要实现所有四个方法。

```
interface Rain {
    void usage();
}
interface RainStorm extends Rain {//暴雨
    void destroy();
}
interface Danger {
    void caution();
}
class WaterFlood implements RainStorm {//洪水
    public void usage() {}
    public void destroy() {}
}
interface MountainTorrents extends RainStorm, Danger {//山洪
    void propertyLoss();
}
```

5.2.4 接口的继承

◆MountainTorrents继承了两个接口，实现MountainTorrents需要实现所有四个方法。

```
class MudrockFlow implements MountainTorrents {//泥石流
    public void usage() {}
    public void destroy() {}
    public void caution() {}
    public void propertyLoss() {}
}
```

◆WaterFlood对象可以隐式转型为：Rain，RainStorm

◆MudrockFlow对象可以隐式转型为：MountainTorrents，Danger，Rain，RainStorm

◆MountainTorrents引用可以隐式转型为：Rain，RainStorm，Danger

5.2.4 接口的继承

◆MountainTorrents继承了两个接口，实现MountainTorrents需要实现所有四个方法。

```
public class E0515HorrorEnv {
    static void u(Rain b) { b.usage(); }
    static void v(RainStorm d) {
        d.usage();
        d.destroy();
    }
    static void w(Danger l) { l.caution(); }
    public static void main(String[] args) {
        RainStorm barney = new WaterFlood();
        u(barney);
        v(barney);
        MountainTorrents vlad = new MudrockFlow();
        u(vlad);
        v(vlad);
        w(vlad);
    }
}
```

5.2.5 适配接口

- 接口最引人注目的原因之一是允许同一个接口有多个实现。它在变成框架中被广泛使用，一般由自己来实现该接口并将自己的对象传递给该方法。
- 下面看一个Scanner的例子。Scanner的构造函数：

```
Scanner(InputStream source)
Scanner(Readable source)
```

- 我们目前为止使用的是第一种(System.in的类型是InputStream)：

```
Scanner scanin = new Scanner(System.in);
```

5.2.5 适配接口

◆Readable接口的定义如下：

```
public interface Readable {
    int read(CharBuffer cb) throws IOException;
}
```

◆任何实现了Readable接口的类，都可以传递给Scanner构造函数，为Scanner提供数据源。

◆read函数的功能要求：从源中读取足够的字符填充传入的CharBuffer，并返回实际读取的字符数量，或者在没有更多字符可读时返回 -1。

◆下面例子实现了Readable接口，并且用随机挑选的字符填充输入缓冲区。

5.2.5 适配接口

◆下面例子实现了Readable接口，并且用随机挑选的字符填充输入缓冲区。

```
import java.nio.*;
import java.util.*;
public class E0516RandomWords implements Readable {
    private static Random rand = new Random();
    private static final char[] capitals =
        "ABCDEFGHIJKLMNOPQRSTUVWXYZ".toCharArray();
    private static final char[] lowers
="abcdefghijklmnopqrstuvwxyz".toCharArray();
    private static final char[] vowels ="aeiou".toCharArray();
    private int count;
    public E0516RandomWords(int count) { this.count = count; }
```

```
    @Override
    public int read(CharBuffer cb) {
        if(count-- == 0)
        return -1; // Indicates end of input
        cb.append(capitals[rand.nextInt(capitals.length)]);
        for(int i = 0; i < 4; i++) {
            cb.append(vowels[rand.nextInt(vowels.length)]);
            cb.append(lowers[rand.nextInt(lowers.length)]);
        }
        cb.append(" ");
        return 10; // 添加的字符数
    }
    public static void main(String[] args) {
        Scanner s = new Scanner(new E0516RandomWords(5));
        while(s.hasNext())
        System.out.println(s.next());
    }
}
```

- 我们实现的read函数没有从设备读取数据，而是随机从数组中选取字符生成输入。
- 输出结果（每次运行不一样）：

Veboqidul

Mamaziuol

Qagajemuu

Murutezaa

Kavehahex

5.2.6 接口中的数据成员

- 接口中的任何成员变量都是 static 和 final 的，接口是创建一组常量值的便捷工具。
- 对于具有常量初始值设定项的 static finals，Java 风格使用全部大写字母（用下划线分隔单个标识符中的多个单词）。
- 接口中的成员变量是自动设置为 public，因此不需要明确指定。
- 这些变量都是public static final:

```
public interface Months {
    int JANUARY = 1, FEBRUARY = 2, MARCH = 3,
    APRIL = 4, MAY = 5, JUNE = 6, JULY = 7,
    AUGUST = 8, SEPTEMBER = 9, OCTOBER = 10,
    NOVEMBER = 11, DECEMBER = 12;
}
```

5.2.7 接口中成员变量的初始化

◆接口中定义的字段不能是“blank finals”，也就是必须在声明时初始化。可以使用非常量表达式进行初始化

```
import java.util.*;
public interface RandVals {
    Random RAND = new Random(47);
    int RANDOM_INT = RAND.nextInt(10);
    long RANDOM_LONG = RAND.nextLong() * 10;
    float RANDOM_FLOAT = RAND.nextFloat() * 10;
    double RANDOM_DOUBLE = RAND.nextDouble() * 10;
}
```

5.2.7 接口中成员变量的初始化

◆接口中定义的字段不能是“blank finals”，也就是必须在声明时初始化。可以使用非常量表达式进行初始化

```
import java.util.*;
public interface RandVals {
    Random RAND = new Random(47);
    int RANDOM_INT = RAND.nextInt(10);
    long RANDOM_LONG = RAND.nextLong() * 10;
    float RANDOM_FLOAT = RAND.nextFloat() * 10;
    double RANDOM_DOUBLE = RAND.nextDouble() * 10;
}
```

◆由于成员变量是静态的，它们在类第一次加载时被初始化。

◆这些值存储在该接口的静态存储区域中。

5.2.8 接口的嵌套

◆在Java中，接口可以嵌套在类或者其他接口中。当一个接口嵌套在另一个类或接口中时，它可以被声明为public、protected、private，或者使用默认访问级别，这是与顶级（非嵌套）接口的区别，因为顶级接口只能使用public或默认访问级别。

◆嵌套接口的主要目的是逻辑上将接口与它所服务的类或接口紧密关联起来。嵌套接口通常只对包含它的类或接口意义重大，而不适合作为一般用途的接口。

5.2.8 接口的嵌套

◆嵌套接口的特点

- ■可见性：嵌套接口的可见性可以受到外部类或接口的限制。在类中可以声明为 private，而在接口中总是 public static。
- ■实现：嵌套接口可以由任何独立的类实现，也可以由外围类或其他任何类实现。
- ■访问：如果嵌套接口被定义为 public，那么它可以在任何地方被访问。如果它有其他访问级别（如 private），那么它的访问将被限制在定义它的外部类中。

5.2.8 接口的嵌套

◆接口中嵌套接口：接口内部可以嵌套另一个接口。嵌套接口默认是public static的，因为所有接口成员（包括嵌套的接口）都自动是public和static的。

```
public interface OuterInterface {
    int outerMethod();

    interface NestedInterface {
        void nestedMethod();
    }
}
```

◆OuterInterface 中嵌套了一个 NestedInterface。如果别的类实现 NestedInterface 必须使用 OuterInterface.NestedInterface来引用这个嵌套接口。

5.2.8 接口的嵌套

◆类中嵌套接口：在类中，一个嵌套接口可以用作定义与外部类紧密相关的协定或回调，而不愿在更广泛的上下文中使用它。

```
public class OuterClass {
    private int value = 100;
    public interface InnerInterface {
        int getValue();
    }
    public class InnerClass implements
InnerInterface {
        public int getValue() {
            return value; // Direct access to
OuterClass's private field 'value'
        }
    }
}
```

5.2.8 接口的嵌套

◆类中嵌套接口：

◆在这个例子中，OuterClass 中嵌套了一个 InnerInterface，它被内部类 InnerClass 实现。这种方式允 许任何类，不仅仅是外部类 OuterClass，实现这个接口，但接口自身与 OuterClass 紧密相连，并且通常只在这个外围上下文中有用。

◆嵌套接口被认为是外围类或接口的一部分，并可以根据需要公开或隐藏。当公开时，嵌套接口可以被任何看得到它的代码实现；当隐藏时（如被声明为 private），它只能被其外围类或外围接口内的代码实现。

5.2.8 接口的嵌套

- 嵌套接口可以用于多种场合，包括：
  - 定义回调，这对于事件监听模式很有用。
  - 为外部类创建分组的专门接口，有助于维护代码结构和可读性。
  - 限制接口的使用范围，使其只在某些特定的上下文中可用，以促进封装。
- 总之，嵌套接口提供了一种优雅的机制，以逻辑上将接口与使用它们的类或外围接口关联起来，实现关注点的分离和代码的模块化。

5.3 内部类*

- 内部类是一种强大的机制，可以把类隐藏在另一个类的内部，用于处理更加复杂的结构而不会污染类的外部接口。
- 内部类可以与外部类进行通信，另外，使用内部类可以使代码具有易读和易维护。
- 内部类又分为几种：
  - 成员内部类（Non-static nested class）：成员内部类是最常见的内部类类型，它不带有 static 关键字。它可以无限制地访问外围类的所有成员，包括私有成员。每个成员内部类的实例都绑定到一个外围类的实例上。
  - 静态内部类（Static nested class）:静态内部类是被 static 修饰的内部类。它可以访问外围类的静态成员和方法，无法直接访问非静态的数据和方法。它通常当作顶层类（top-level class）的一个成员。

5.3 内部类*

- 内部类又分为几种：
  - 局部内部类：局部内部类是在一个块内部（如一个方法或作用域块）定义的类，它只在该块的内部可见。
  - 匿名内部类：匿名内部类是没有名字的局部内部类。它用于那些需要的类实现或扩展另一个类或接口的地方。很多时候匿名内部类用于实现接口或者继承某个类的临时需求。匿名内部类非常适合于那些简单的扩展或实现操作，通常它们都只用一次。

◆类的内部使用内部类，就和使用外部类一样。

```
public class E0518Mail {
    class Contents {
        private String i = "BB";
        public String text() { return i; }
    }
    class Receiver {
        private String address;
        Receiver(String whereTo) {
            address = whereTo;
        }
        String readAddress() { return address; }
    }
```

◆类的内部使用内部类，就和使用外部类一样。

```
    // 使用内部类就像使用 Mail 中的其他类
    public void send(String dest) {
        Contents c = new Contents();
        Receiver d = new Receiver(dest);
        System.out.println(d.readAddress());
    }
    public static void main(String[] args) {
        E0518Mail p = new E0518Mail();
        p.send("cs@nwpu.edu.cn");
    }
}
```

5.3.1 内部类的创建

◆外部类方法可以返回对内部类的引用，如 to()和 contents()方法（E0519Mail.java）

```
public Contents contents() {
    return new Contents();
}
public void send(String dest) {
    Contents c = contents();
    Receiver d = to(dest);
    System.out.println(d.readAddress());
}
```

5.3.1 内部类的创建

◆如果想从外部类的非静态方法之外的任意位置创建某个内部类的对象，那么必须像在main()方法中那样，具体地指明这个对象的类型：*OuterClassName.InnerClassName*。

◆在外部类的静态方法中也可以直接指明类型*InnerClassName*，在其他类中需要指明*OuterClassName.InnerClassName*。）

```
public static void main(String[] args) {
    E0519Mail p = new E0519Mail();
    p.send("cs@nwpu.edu.cn");
    E0519Mail q = new E0519Mail();
    // 定义内部类的引用:
    E0519Mail.Contents c = q.contents();
    E0519Mail.Receiver d = q.to("is@nwpu.edu.cn");
}
```

5.3.2 外部类的链接

- 内部类与外部类对象的关联：非静态内部类（也称为实例内部类）持有对创建它的外部类实例的引用。这意味着内部类可以直接访问外部类的实例成员，包括私有成员。这是内部类的特权，因为它被视作外部类的一部分。
- 内部类的对象可以直接访问外部类成员变量和方法，不需要任何特殊的语法结构或限定符。
- 成员变量命名冲突：如果内部类和其外部类有相同名称的成员，内部类默认访问其自身的成员。如果需要访问外部类的同名成员，可以使用 OuterClass.this.memberName 这样的语法结构。
- 静态内部类：对于静态内部类（也称为嵌套类），情况有所不同。静态内部类不持有对外部类实例的引用，所以它不能直接访问外部类的非静态成员。它只能通过一个外部类的对象引用或访问外部类的静态成员

5.3.2 外部类的链接

- 迭代器模式的例子： E0520Sequnce内部有一个Object数组，利用add追加对象到数组中。
- Selector接口提供操作一个序列的功能：end, current,next

```
interface Selector {
    boolean end();
    Object current();
    void next();
}
public class E0520Sequnce {
    private Object[] items;
    private int next = 0;
    public E0520Sequnce(int size) { items = new Object[size]; }
    public void add(Object x) {
        if(next < items.length)
        items[next++] = x;
    }
```

5.3.2 外部类的链接

- 私有类SequenceSelector 实现Selector接口，把内部的数组作为序列操作。
- 由于SequenceSelector是类的成员，可以操作所有外部类的方法和变量，包括private变量items.
- selector()利用Selector类型的SequenceSelector实例。

```
private class SequenceSelector implements Selector {
    private int i = 0;
    public boolean end() { return i == items.length; }
    public Object current() { return items[i]; }
    public void next() { if(i < items.length) i++; }
}
public Selector selector() {
    return new SequenceSelector();
}
```

5.3.2 外部类的链接

◆外部类取得Selector接口实例后，利用这个实例遍历序列。

```
public static void main(String[] args) {
    E0520Sequnce sequence = new E0520Sequnce(10);
    for(int i = 0; i < 10; i++)
        sequence.add(Integer.toString(i));
    Selector selector = sequence.selector();
    while(!selector.end()) {
        System.out.print(selector.current() + " ");
        selector.next();
    }
}
```

5.3.2 外部类的链接

◆非静态内部类对象的构造需要对外部类对象的引用，如果无法访问该引用，编译器会报错。 。

```
public class OuterClass {
    private int outerValue = 100;
    class InnerClass {
        public void display() {
            System.out.println("Outer value: " + outerValue);
        }
    }
}
public class TestClass {
    public static void main(String[] args) {
        OuterClass.InnerClass innerObject;
        innerObject = new OuterClass().new InnerClass(); // 正确的构造方式
        // 下面这种尝试在不引用外部类实例的情况下创建内部类对象的方法会报错
        // innerObject = new OuterClass.InnerClass(); // 编译错误
    }
}
```

5.3.3 使用.this 和.new 操作

◆如果你需要生成对外部类对象的引用，可以使用外部类的名字后面紧跟圆点和this。这样产生的引用自动地具有正确的类型，这一点在编译期就被知晓并受到检查，因此没有任何运行时开销。

```
public class E0520DoThis {
    void f() { System.out.println("DotThis.f()"); }
    public class Inner {
        public E0520DoThis outer() {
            return E0520DoThis.this;
            //不写外部类名"this"时，表示为内部类对象"this"
        }
    }
    public Inner inner() { return new Inner(); }
    public static void main(String[] args) {
        E0520DoThis dt = new E0520DoThis();
        E0520DoThis.Inner dti = dt.inner();
        dti.outer().f();
    }
}
```

5.3.3 使用.this 和.new 操作

- 前一页例子使用了内部方法inner()创建内部类对象。
- 当其它类需要一个外部类的内部类对象时，必须使用.new 语法，在 new 表达式中提供了另外一个类对内部类对象的引用.

```
public class DotNew {
    public class Inner {}
    public static void main(String[] args) {
        DotNew dn = new DotNew();
        DotNew.Inner dni = dn.new Inner();
    }
}
```

5.3.3 使用.this 和.new 操作

◆嵌套类（静态内部类）对象的创建，不需要对外部类对象的引用。

```
public class OuterClass {
    // 静态嵌套类
    static class StaticNestedClass {
        void display() {
            System.out.println("Inside static nested class.");
        }
    }
}
public class TestClass {
    public static void main(String[] args) {
        // 直接使用外部类名来创建静态嵌套类对象
        OuterClass.StaticNestedClass nestedObject = new
OuterClass.StaticNestedClass();
        nestedObject.display();
    }
}
```

5.3.4 内部类向上转换类型

- 当将内部类向上转型为其基类，尤其是转型为一个接口的时候，内部类就有了用武之地。
- 这是因为此内部类某个接口的实现能够完全不可见，并且不可用。所得到的只是指向基类或接口的引用，所以能够很方便地隐藏实现细节。
- 为了演示首先定义两个接口:

```
interface Receiver {
    String readAddress();
    }
interface Contents{
    String text();
}
```

5.3.4 内部类向上转换类型

◆Pcontents和Preceiver创建的对象，分别以接口类型返回。

◆这两个类自身是设有访问保护的： private和protected

```
class Mail {
    private class PContents implements Contents {
        private String i = "BB";
        public String text() { return i; }    }
    protected class PReceiver implements Receiver {
        private String address;
        private PReceiver(String whereTo) {
            address = whereTo;
        }
        public String readAddress() { return address; }    }
    public Receiver receiver(String s) {
        return new PReceiver(s);    }
    public Contents contents() {
        return new PContents();    }
}
```

5.3.4 内部类向上转换类型

◆调用者不必关心对象的真正类型，只需要通过接口引用来操作返回的对象就可以了。

```
public class E0521Mail {
    public static void main(String[] args) {
        Mail p = new Mail();
        Contents c = p.contents();
        Receiver d = p.receiver("cs@nwpu.edu.cn");
        // 以下语句非法，不能访问私有类:
        //! Mail4.PContents pc = p.new PContents();
    }
}
```

5.3.5 方法和作用域中的内部类

- 可以在一个方法里面或者在任意的作用域内定义内部类。这么做有两个理由：
- 1.如前所示，你实现了某类型的接口，于是可以创建并返回对其的引用。
- 2.你要解决一个复杂的问题，想创建一个类来辅助你的解决方案，但是又不希望这个类是公共可用的。

5.3.5 方法和作用域中的内部类

◆在方法范围内（而不是另一个类的范围）创建整个类。这称为本地内部类。 Preceiver的作用域在方法内部；利用向上转型把对象作为接口返回。

```
public class E0522Mail {
    public Receiver receiver(String s) {
        class PReceiver implements Receiver {
            private String address;
            private PReceiver(String whereTo) {
                address = whereTo;
            }
            public String readAddress() { return address; }
        }
        return new PReceiver(s);
    }
    public static void main(String[] args) {
        E0522Mail p = new E0522Mail();
        Receiver d = p.receiver("cs@nwpu.edu.cn");
    }    }
```

5.3.5 方法和作用域中的内部类

◆代码块中的内部类，作用域在代码块内部。(E0523Mail.java)

```
。。。。。。
    private void internalTracking(boolean b) {
        if(b) {
            class TrackingSlip {
                private String id;
                TrackingSlip(String s) {
                    id = s;
                }
                String getSlip() { return id; }
            }
            TrackingSlip ts = new TrackingSlip("slip");
            String s = ts.getSlip();
        }
        // 范围越界，不能在这里使用:
        //! TrackingSlip ts = new TrackingSlip("x");
    }
。。。。。。
```

5.3.6 匿名内部类

- 匿名内部类是一种没有名称的内部类。它们通常用于创建一次性使用的类实例，当你只需要使用一个类的实例一次时，就可以使用匿名内部类，而不需要实际去命名一个类。
- 匿名内部类有以下几个特点：
  - 没有类名，只能一次性使用：由于它没有名称，所以你不能在声明它之外的任何地方再次创建其实例。
  - 用在即时需求的场合：如果你需要扩展一个类或接口但又不想创建一个子类，匿名内部类便非常有用。最常见的场合是在处理事件监听和回调接口时。
  - 可以访问外部类的成员：匿名内部类可以访问定义它的作用域内的数据，包括最终（final）或有效最终（effectively final）的局部变量。
  - 可实现接口或继承类：匿名内部类可以实现一个接口或继承一个类。但和正常的类不同，一个匿名内部类只能实现一个接口或继承一个类。

◆利用扩展接口创建匿名内部类（E0524Mail.java）:

```
interface Contents {
    String text();
}
public class E0524Mail {
    public Contents contents() {
        return new Contents() { //插入一个内部类定义
            private String i = "BB";
            @Override
            public String text() { return i; }
        }; //“;”不能遗漏
    }
    public static void main(String[] args) {
        E0524Mail p = new E0524Mail();
        Contents c = p.contents();
    }
}
```

5.3.6 匿名内部类

◆相当于以下命名内部类的简写:

```
public class Mail7b {
    class MyContents implements Contents {
        private String i = "BB";
        public String text() { return i; }

    }
    public Contents contents() { return new MyContents(); }
    public static void main(String[] args) {
        Mail7b p = new Mail7b();
        Contents c = p.contents();
    }
}
```

5.3.6 匿名内部类

◆传递参数给内部类构造方法

执行结果：AACC

```
class Wrapping {
    String v;
    public Wrapping(String x) { v = x; }
    public String text() { return v; }
} ///:~
public class E0525Mail {
    public Wrapping wrapping(String x) {
        // 基类构造函数的调用:
        return new Wrapping(x) { //传递参数给构造函数.
            public String text() {
                return super.text() + "CC";
            }
        }; //这里的分号“;”不能忽略
    }
    public static void main(String[] args) {
        E0525Mail p = new E0525Mail();
        Wrapping w = p.wrapping("AA");
        System.out.println(w.text());
    }
}
```

5.3.6 匿名内部类

◆匿名类定义中，对字段执行初始化：内部类中要使用一个在外部类（匿名内部类之外）定义的对象，编译器要求参数引用是final

```
interface Receiver {
    String readAddress();
}
public class E0526Mail {
    //匿名内部类的参数，其修饰符必须为 final:
    public Receiver receiver(final String dest) {
        return new Receiver() {
            private String address = dest;
            public String readAddress() { return address; }
        };
    }
    public static void main(String[] args) {
        E0526Mail p = new E0526Mail();
        Receiver d = p.receiver("cs@nwpu.edu.cn");
    }
}
```

5.3.6 匿名内部类

◆通过实例初始化（块），为匿名内部类创建构造函数。

```
abstract class Base {
    public Base(int i) {
        System.out.println("Base constructor, i = " + i);    }
    public abstract void f();
}
public class E0527AnonymousConstrutor {
    public static Base getBase(int i) {
        return new Base(i) { //i没有真正使用，可以不是final
            { System.out.print("Inside instance initializer"); }
            public void f() {
                System.out.print("In anonymous f()");
            }
        };
    }
    public static void main(String[] args) {
        Base base = getBase(47);
        base.f();
    }    }
```

```
Base constructor, i = 47
Inside instance initializer
In anonymous f()
```

5.3.6 匿名内部类

◆dest和price在匿名类内部都用到了，所以他们必须是final。

```
public class Mail10 {
    public Receiver receiver(final String dest, final float price) {
        return new Receiver() {
            private int cost;
            // 实例初始化:
            {    cost = Math.round(price);
                 if(cost > 100)
                    System.out.println("Over budget!");
            }
            private String address = dest;
            public String readAddress() { return address; }
        };
    }
    public static void main(String[] args) {
    Mail10 p = new Mail10();
    Receiver d = p.receiver("cs@nwpu.edu.cn", 101.395F);
    }
}
```

5.3.6 匿名内部类

- 对于匿名类而言，实例初始化块的实际效果就是构造器。
- 当然它受到了限制你不能重载实例初始化方法，所以你仅有一个这样的构造器。

5.3.7 嵌套类

- 静态内部类又叫嵌套类(Static Nested Class)
- 普通内部类的对象隐式地保留对创建它的外部类对象的引用，当内部类是静态的时，内部类的对象就不存在对外部类的对象引用。
- 嵌套类意味着：
  - ①不需要外部类对象来创建内部类（嵌套类）的对象。
  - ②不能从嵌套类的对象访问非静态外部类对象
- 嵌套类在另一个方面也不同于普通的内部类。普通内部类中的字段和方法定义不能超越外部类（对象）的层次，所以普通内部类不能有静态数据、静态字段或嵌套类。但是，嵌套类可以具有所有这些

◆举例(E0528Mail.java)：在 main( )中，不需要外部类 E0528Mail 的对象，使用静态成员的调用来返回对 Contents 和Receiver 的引用。

```
public class E0528Mail {
    private static class MailContents implements Contents {
      。。。。。。                    }
   protected static class MailReceiver implements Receiver {
      。。。。。。                    }
    public static Receiver receiver(String s) {
        return new MailReceiver(s);     }
    public static Contents contents() {
        return new MailContents();     }
    public static void main(String[] args) {
        Contents c = contents();
        Receiver d = receiver("cs@nwpu.edu.cn");
    }
}
```

5.3.8 接口内的类

- 嵌套类可以是接口的一部分。
- 放在接口中的任何类都自动是公共的和静态的。
- 由于类是静态的，它不会违反接口的规则，嵌套类可以放置在接口的命名空间内。甚至可以在内部类中实现接口中的函数.

5.3.8 接口内的类

- 当要创建一些通用代码以用于该接口的所有不同实现时，将类嵌套放置在接口内很方便。
- 运行方法： java E0529ClassInInterface$Test

```
public interface E0529ClassInInterface {
    void howdy();
    class Test implements E0529ClassInInterface {
        public void howdy() {
            System.out.println("Howdy!");
        }
        public static void main(String[] args) {
            new Test().howdy();
        }
    }
}
```

运行结果:Howdy!

5.3.9 从多重嵌套类向外延伸

- 内部类嵌套的深度无关紧要，它可以透明地访问嵌套在其中的所有类的所有成员
- 下面例子MNA.A.B中，调用方法g()和f()不需要任何条件（即使它们被定义为private）

```
//内部类可以访问它们所嵌套的类的所有级别的所有成员.
class MNA {
    private void f() {}
    class A {
        private void g() {}
        public class B {
            void h() {
                g();
                f();
            }
        }
    }
}
```

5.3.9 从多重嵌套类向外延伸

- 例子还展示了如何从不同的类里创建多层嵌套的内部类对象的基本语法。
- “.new”语法能产生正确的作用域，所以不必在调用构造器时限定类名。

```
public class E0530MultiNestingAccess {
    public static void main(String[] args) {
        MNA mna = new MNA();
        MNA.A mnaa = mna.new A();
        MNA.A.B mnaab = mnaa.new B();
        mnaab.h();
    }
}
```

5.3.10 内部类的用途

- 使用内部类最吸引人的原因是：每个内部类都能独立地继承自一个（接口的）实现，所以无论外围类是否已经继承了某个（接口的）实现，对于内部类都没有影响。
- 接口只解决了多继承的部分问题，但是内部类有效地允许“多重实现继承”。也就是说，内部类有效地允许代码实现从多个具体类和抽象类（非接口）继承
- 比如： 两个接口必须以某种方式在一个类中实现。由于接口的灵活性，有两种选择：单个类或内部类
- 如果有抽象或具体类而不是接口，类必须以某种方式实现其他两个类，那么会只能使用内部类

5.3.10 内部类的用途

- 内部类还能满足其他特性：
  - ①内部类可以有多个实例，每个实例都有自己的状态信息，这些信息独立于外部类对象中的信息。
  - ②在单个外部类中，可以有多个内部类，每个内部类都实现相同的接口或以不同的方式从同一类继承。
  - ③内部类对象的创建点与外部类对象的创建无关。
  - ④与内部类没有潜在的混淆“is-a”关系；它是一个单独的实体。

5.3.11 GUI 中的回调函数

- 闭包（closure）是一个可调用的对象，它保留了创建它的作用域内的信息。
- 内部类是一个面向对象的闭包，因为它不仅包含来自外部类对象的每一条信息（“它被创建的范围”），而且它自动持有对整个外部类对象的引用，它有权操作所有成员，甚至是私有成员。
  - 在Java 8之后，可以使用lambda表达式来生成闭包行为，并且语法更加精细和简洁；
- 回调（callback)是一种常见的编程模式，用于处理异步事件或定义在特定动作之后需要执行的代码。最典型的例子是事件监听器，当某个事件（如用户点击按钮）发生时，监听器对象会被调用以执行相应操作。

5.3.11 GUI 中的回调函数

- 回调处理时，需要使用某些原始对象的信息。
- 在C/C++中，回调通过指针实现。而Java通过内部类很好地解决了这个问题，同时避免了指针的使用。
- 举例：E0531Callbacks.java
- Callee1使用外部类实现了简单的回调。
- Callee2继承自MyIncrement。Calle2为了实现自己的回调，同时又不影响继承来的increment()，使用内部类独立地实现了Incrementable接口。
- 在Callee2中除了getCallbackReference()以外，其他成员都是private的。利用内部类闭包实现了Incrementable，以提供一个返回Callee2的“钩子”（hook）而且是一个安全的钩子。无论谁获得此Incrementable的引用，都只能调用increment()，除此之外没有其他功能

5.3.12 继承一个内部类

◆内部类构造函数必须附加到外部类对象的引用，所以当继承一个内部类继承时，必须初始化对外部类对象的隐式引用，必须使用特殊语法来明确关联

```
//继承内部类.
class WithInner {
    class Inner {}
}
public class InheritInner extends WithInner.Inner {
    //! InheritInner() {} //此构造函数编译错误
    InheritInner(WithInner wi) {
        wi.super();
    }
    public static void main(String[] args) {
        WithInner wi = new WithInner();
        InheritInner ii = new InheritInner(wi);
    } }
```

◆当需要创建一个构造函数时，默认的就不行了，不能只传递一个对封闭对象的引用，必须使用语法：
enclosingClassReference.super();

5.3.13 内部类的覆盖

- 当定义一个内部类，然后从外部类继承并重新定义内部类时，内部类是不会被覆盖的。
- 当从外部类继承时，内部类“覆盖”等操作不会发生。这两个内部类是完全独立的实体，每个实体都在自己的命名空间中。但是，仍然可以从内部类显式继承

5.3.14 本地内部类

- 内部类也可以在代码块内创建，通常是在方法的主体内。本地内部类不能有访问说明符，因为它不是外部类的一部分
- 但它可以访问当前代码块中的最终变量和外部类的所有成员。
- 比较本地内部类和匿名内部类：
  `E0532LocalInnerClass.java`

5.3.15 内部类编译时的标识符

- 内部类也会生成.class 文件以包含其 Class 对象的信息。
- 内部类的名称有一个严格的公式：外部类的名称，后跟一个“$”，然后是内部类的名称。例如，E0532LocalInnerClass.java 创建的.class 文件包括：
  - Counter.class
  - E0532LocalInnerClass$1.class
  - E0532LocalInnerClass$LocalCounter.class
  - E0532LocalInnerClass.class
- 如果内部类是匿名的，编译器会简单地开始生成数字作为内部类标识符。
- 如果内部类嵌套在内部类中，则它们的名称只需附加在“$”和外部类标识符之后。
- 这种生成内部名称的方案简单明了，可以处理大多数情况。由于它是 Java 的标准命名方案，因此生成的文件自动独立于平台。